Python dla Inżynierii Robotyki – Ściąga

Dokument: Kompendium zagadnień z kursu Bro Code (#35–#65), przystosowane do praktyki w robotyce.
Cel: Szybka referencja + best practices dla programisty.
Format: Każdy temat – składnia, przykład, antywzorce, use-case w robotyce.

---

1. ITERABLES – Jak pracować z kolekcjami

Koncept

Iterable to obiekt, który można iterować (przelatać pętlą `for`). Iterator to maszyna, która zwraca elementy jeden po drugim.

```
# Wszystkie są iterable'ami:
lista = [1, 2, 3]
krotka = (1, 2, 3)
string = "ABC"
slownik = {"a": 1, "b": 2}
zbior = {1, 2, 3}

# Jak się tego pozbyć manualnie (rzadko potrzebne, ale warto znać)
iterator = iter(lista)  # Tworzy iterator
print(next(iterator))   # Zwraca: 1
print(next(iterator))   # Zwraca: 2
# Gdy nie ma więcej elementów: StopIteration exception
```

Best Practices

- **Zawsze** iteruj pętlą `for` zamiast `while + next()` – czytelniej, bezpieczniej.
- Użyj `enumerate()` gdy potrzebujesz indeksu i wartości jednocześnie.
- W robotyce: iteruj po listach sensorów, konfigach, punktach trajektorii.

```
# ✓ DOBRZE
sensors = ["lidar", "camera", "imu"]
for idx, sensor in enumerate(sensors):
    print(f"Sensor {idx}: {sensor}")

# × ŹRÓDŹ BŁĘDÓW
iterator = iter(sensors)
while True:
    try:
        sensor = next(iterator)
        print(sensor)
    except StopIteration:
        break
```

Use-case: Robotyka

```
# Iteracja po punktach trajektorii
trajectory = [(0, 0), (1, 0), (1, 1), (0, 1)]
for waypoint in trajectory:
    x, y = waypoint
    move_robot_to(x, y)
```

---

2. MEMBERSHIP OPERATORS – Sprawdzanie przynależności

Koncept

Operatory `in` i `not in` sprawdzają, czy element istnieje w kolekcji.

```
lista = [1, 2, 3]
slownik = {"name": "robot", "type": "humanoid"}
string = "ROS2"

print(2 in lista)                    # True
print("robot" in slownik)            # True (domyślnie sprawdza klucze!)
print("robot" in slownik.values())   # True (jeśli chcesz wartości)
print("O" in string)                 # True

print(5 not in lista)                # True
```

Best Practices

- `in` na liście jest O(n) – powolne dla dużych kolekcji. Użyj `set` dla szybkości O(1).
- Na słownikach `in` sprawdza **klucze**, nie wartości.
- Na stringach `in` sprawdza **podciąg**, nie pojedynczy znak.

```
# ✓ DOBRZE (szybkie sprawdzenie dla dużych zbiorów)
active_motors = {"motor_1", "motor_2", "motor_3"}  # Set, O(1)
if "motor_1" in active_motors:
    activate_motor("motor_1")

# × WOLNE
active_motors = ["motor_1", "motor_2", "motor_3"]  # Lista, O(n)
if "motor_1" in active_motors:  # Przeszukuje całą listę
    activate_motor("motor_1")
```

Use-case: Robotyka

```
# Sprawdzenie, czy sensor jest w konfiguracji
available_sensors = {"lidar", "camera", "imu", "pressure"}
required_sensor = "lidar"

if required_sensor in available_sensors:
    print(f"Sensor {required_sensor} is available")
else:
    raise ValueError(f"{required_sensor} not in configuration!")
```

---

3. LIST COMPREHENSIONS – Filtrowanie i mapowanie list (O MEGA WAŻNE!)

Koncept

Compact sposób na tworzenie nowych list przez filtrowanie lub transformację istniejących.

```
# Zwykły sposób (długi):
numbers = [1, 2, 3, 4, 5]
squared = []
for n in numbers:
    squared.append(n ** 2)

# List comprehension (Pythonicka):
squared = [n ** 2 for n in numbers]

# Z filtrem:
even_squared = [n ** 2 for n in numbers if n % 2 == 0]

# Zagnieżdżona:
matrix = [[1, 2], [3, 4], [5, 6]]
flattened = [elem for row in matrix for elem in row]  # [1, 2, 3, 4, 5, 6]
```

Best Practices

- **Zawsze** używaj comprehension zamiast `append()` w pętli – szybciej (~2x) i czytelniej.
- Nie zagnieżdżaj ponad 2–3 poziomy – traci się czytelność. Wtedy użyj zwykłej pętli.
- Do słowników: `dict comprehension`, do setów: `set comprehension`.

```
# Słownik comprehension
sensor_readings = [("lidar", 5.2), ("camera", True), ("imu", [0.1, 0.2,
0.3])]
sensor_dict = {name: value for name, value in sensor_readings}
# {"lidar": 5.2, "camera": True, "imu": [0.1, 0.2, 0.3]}

# Set comprehension (unikalne wartości)
joint_angles = [90, 45, 90, 180, 45]
```

```
unique_angles = {angle for angle in joint_angles}
# {45, 90, 180}
```

Use-case: Robotyka

```
# Filtrowanie sensorów spoza zakresu bezpieczeństwa
sensor_values = {"lidar": 0.5, "camera": True, "pressure": 150, "imu":
0.05}
min_distance = 0.3

# Znajdź wszystkie sensory z dystansem poniżej minimum
dangerous = {name: value for name, value in sensor_values.items()
             if isinstance(value, float) and value < min_distance}

if dangerous:
    print(f"WARNING: Obstacles detected! {dangerous}")
    trigger_emergency_stop()
```

---

4. MATCH-CASE STATEMENTS – Warunkowe rozgałęzienia (Python 3.10+)

Koncept

`match-case` to nowoczesny odpowiednik `switch` z C/Java. Bardziej czytany niż seria `if-elif-else`.

```
robot_state = "emergency"

match robot_state:
    case "idle":
        print("Robot waiting for commands")
    case "moving":
        print("Robot in motion")
    case "emergency":
        print("EMERGENCY STOP activated!")
    case _:  # domyślnie (like default)
        print("Unknown state")
```

Zaawansowane: Pattern matching

```
# Rozpoznawanie typów i wartości
sensor_data = ("lidar", 0.5)  # tuple (name, distance)

match sensor_data:
    case ("lidar", distance) if distance < 0.3:
        print(f"Obstacle too close: {distance}m")
```

```
    case ("camera", _):  # _ = ignoruj wartość
        print("Camera detected something")
    case ("imu", x, y, z):  # rozpakowanie
        print(f"IMU: X={x}, Y={y}, Z={z}")
    case _:
        print("Unknown sensor")
```

Best Practices

- Używaj `match-case` gdy masz wiele wariantów stanu (finite state machine w robotyce).
- Guard (`if`) pozwala na dodatkową walidację warunku.
- `_` zastępuje każdą wartość (wildcard).

```
# Antywzorzec: zbyt wiele if-elif
if action == "move_forward":
    move(1.0)
elif action == "move_backward":
    move(-1.0)
elif action == "rotate_left":
    rotate(-90)
elif action == "rotate_right":
    rotate(90)
else:
    pass

# ✓ DOBRZE: match-case
match action:
    case "move_forward":
        move(1.0)
    case "move_backward":
        move(-1.0)
    case "rotate_left":
        rotate(-90)
    case "rotate_right":
        rotate(90)
    case _:
        pass
```

Use-case: Robotyka

```
# Finite State Machine dla robota
def process_command(cmd_type, cmd_data):
    match (cmd_type, cmd_data):
        case ("move", (x, y, z)):
            move_to_position(x, y, z)
        case ("grip", force) if force > 0:
            gripper_open()
        case ("grip", force) if force == 0:
            gripper_close()
```

```
        case ("calibrate", axis):
            calibrate_joint(axis)
        case ("stop", _):
            emergency_stop()
        case _:
            log_error(f"Unknown command: {cmd_type}")
```

---

5. MODULES – Organizacja kodu w projekcie

Koncept

Moduł to plik `.py`. Import pozwala na ponowne użycie kodu z innych plików i logiczną organizację.

```
project/
├── main.py
├── robot_controller.py
├── sensors.py
└── utils.py
```

Składnia importu

```
# Import całego modułu
import robot_controller
robot_controller.move_forward()

# Import konkretnej funkcji/klasy
from robot_controller import move_forward, Robot
move_forward()

# Import z aliasem
from robot_controller import move_forward as move_fw
move_fw()

# Import wszystkiego (✗ UNIKAJ!)
from robot_controller import *  # Zanieczyszcza namespace, niebezpieczne
```

Best Practices

- **Jeden moduł = jedna odpowiedzialność** (SRP – Single Responsibility Principle).
- Nazwa modułu: małe litery, `snake_case`, bez spacji.
- Unikaj **cyklicznych importów** (modul A importuje B, B importuje A).
- Pliki konfiguracyjne (np. `config.py`) na górze.

```
project/
├── main.py              # Entry point
```

```
├── config.py            # Parametry globalne
├── hardware/
│   ├── __init__.py      # Czyni folder modułem
│   ├── motor.py
│   └── sensor.py
├── controller/
│   ├── __init__.py
│   └── robot_logic.py
└── utils/
    ├── __init__.py
    ├── logger.py
    └── math_helpers.py
```

```python
# main.py
from config import ROBOT_MODEL, MAX_SPEED
from hardware.motor import Servo
from controller.robot_logic import RobotController
from utils.logger import setup_logging

setup_logging()
robot = RobotController(model=ROBOT_MODEL)
robot.start()
```

Use-case: Robotyka

```
ros_pkg/
├── scripts/
│   └── main.py
├── src/
│   ├── robot_node.py        # ROS node
│   ├── hardware/
│   │   ├── motor_driver.py   # Sterownik silników
│   │   └── sensor_reader.py  # Czytnik sensorów
│   └── algorithms/
│       └── path_planner.py   # Planowanie trajektorii
```

---

6. SCOPE RESOLUTION – Zmienne globalne vs lokalne (LEGB rule)

Koncept

Gdzie Python szuka zmiennej? Hierarchia: **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in.

```python
x = 10  # Global

def outer():
    y = 20  # Enclosing (dla inner)
```

```
    def inner():
        z = 30  # Local
        print(x)  # Szuka: local? Nie. Enclosing? Nie. Global? Tak! Zwraca
10.
        print(y)  # Szuka: local? Nie. Enclosing? Tak! Zwraca 20.
        print(z)  # Szuka: local? Tak! Zwraca 30.

    inner()

outer()

# print(y)  # × NameError – y jest zmienna lokalną w outer(), nie widać z
globalnego scope
```

Słowo kluczowe `global` (unikaj!)

```
counter = 0

def increment():
    global counter  # Mówi: "counter odnosi się do globalnej zmiennej"
    counter += 1

increment()
print(counter)  # 1

# × ANTYWZORZEC: Zmienne globalne są trudne do debugowania.
# ✓ Zamiast tego: zwróć wartość z funkcji lub użyj klasy.
```

Best Practices

- **Unikaj** zmiennych globalnych. Są źródłem bugów i trudne do testowania.
- Używaj `global` **tylko** w małych skryptach (debugging, config).
- W poważnych projektach: klasy i funkcje ze zwrotami wartości.

```
# × ŹRÓDŁO BUGÓW
robot_state = None

def move_robot():
    global robot_state
    robot_state = "moving"

def stop_robot():
    global robot_state
    robot_state = "stopped"

# ✓ DOBRZE: Klasa przechowuje stan
class Robot:
    def __init__(self):
```

```
        self.state = "idle"

    def move(self):
        self.state = "moving"

    def stop(self):
        self.state = "stopped"

robot = Robot()
robot.move()
print(robot.state)  # "moving"
```

Use-case: Robotyka

```
# × Złe: globalne parametry
MAX_SPEED = 10
PORT = "/dev/ttyUSB0"

def init_motor():
    global MAX_SPEED
    MAX_SPEED = 15  # Ktoś zmienił!

# ✓ Dobre: config moduł + klasa
# config.py
ROBOT_CONFIG = {
    "max_speed": 10,
    "port": "/dev/ttyUSB0",
    "timeout": 1.0
}

# robot.py
from config import ROBOT_CONFIG

class Motor:
    def __init__(self, config):
        self.max_speed = config["max_speed"]
        self.port = config["port"]
```

---

7. if __name__ == '__main__' – Entry point pattern

Koncept

Pozwala na różne zachowanie pliku w zależności od tego, czy jest uruchomiony bezpośrednio czy importowany.

```
# motor.py
def spin(speed):
    print(f"Motor spinning at {speed} RPM")
```

```
if __name__ == "__main__":
    # Ten kod uruchamia się TYLKO gdy uruchomisz: python motor.py
    spin(100)
    print("Motor test complete")

    # Ale gdy zrobisz: from motor import spin
    # Ten blok się NIE wykonuje
```

Logika

- `__name__` to wbudowana zmienna, której wartość zależy od kontekstu.
- Gdy plik uruchomisz bezpośrednio: `__name__ == "__main__"`
- Gdy plik zaimportujesz: `__name__ == "motor"` (nazwa modułu)

```
# main.py
from motor import spin

spin(150)  # Nie będzie "Motor test complete" – __main__ blok się nie
wykonał
```

Best Practices

- **Zawsze** używaj tego wzorca w plikach, które mają być modułami.
- W bloku `if __name__` umieszczaj kod testowy, demonstracyjny czy setupowy.
- To ułatwia testowanie i debugowanie.

```
# robot.py
class Robot:
    def __init__(self, name):
        self.name = name

    def start(self):
        print(f"{self.name} started")

if __name__ == "__main__":
    # Test lokalny
    robot = Robot("TestRobot")
    robot.start()
    print("Local test passed")

# main.py
from robot import Robot

robot = Robot("ProductionRobot")
robot.start()  # Nie będzie "Local test passed"
```

Use-case: Robotyka

```
# motor_driver.py
class Servo:
    def __init__(self, pin):
        self.pin = pin

    def set_angle(self, angle):
        print(f"Setting servo on pin {self.pin} to {angle}°")

if __name__ == "__main__":
    # Test sterownika bezpośrednio
    servo = Servo(pin=11)
    servo.set_angle(90)
    servo.set_angle(180)
    print("Servo calibration complete")

# robot_main.py
from motor_driver import Servo

# Import nie spowoduje testowego kod – bezpieczne!
servo = Servo(pin=11)
servo.set_angle(45)
```

---

8. OBJECT-ORIENTED PROGRAMMING (OOP) – Fundamenty robotyki

8.1 Klasy i obiekty – Podstawa

```
class Motor:
    """Reprezentacja silnika w robocie"""

    def __init__(self, name, max_rpm):
        """Konstruktor – wykonuje się przy utworzeniu obiektu"""
        self.name = name           # Zmienna instancji
        self.max_rpm = max_rpm
        self.current_rpm = 0

    def spin(self, rpm):
        """Metoda – funkcja dostępna dla obiektu"""
        if rpm > self.max_rpm:
            raise ValueError(f"RPM {rpm} exceeds max {self.max_rpm}")
        self.current_rpm = rpm
        print(f"{self.name} spinning at {rpm} RPM")

    def stop(self):
        self.current_rpm = 0
        print(f"{self.name} stopped")

# Tworzenie obiektów (instancji)
```

```
motor1 = Motor("Left wheel", 200)
motor2 = Motor("Right wheel", 200)

motor1.spin(150)  # Left wheel spinning at 150 RPM
motor2.spin(150)

# `self` oznacza "ten konkretny obiekt"
# motor1.spin(150) wewnętrznie: Motor.spin(motor1, 150)
```

Best Practices – Klasy

- Nazwa klasy: `PascalCase` (pierwsza litera wielka).
- Inicjalizuj zmienne w `__init__`, nie gdzie indziej.
- Dokumentuj klasy i metody za pomocą docstringów (`"""..."""`).

```
# ✓ DOBRZE
class Robot:
    """Main robot controller class."""

    def __init__(self, model):
        self.model = model
        self.motors = []

    def add_motor(self, motor):
        """Add a motor to the robot."""
        self.motors.append(motor)

# ✗ ŹRÓDŁO BUGÓW
class robot:
    def __init__(self):
        pass

    def add_motor(motor):  # Brakuje self!
        pass
```

8.2 Zmienne instancji vs zmienne klasy

```
class Robot:
    robot_count = 0  # Zmienna klasy – współdzielona przez WSZYSTKIE
instancje

    def __init__(self, name):
        self.name = name  # Zmienna instancji – unikalna dla każdego
obiektu
        Robot.robot_count += 1

robot1 = Robot("R1")
robot2 = Robot("R2")
```

```
print(robot1.name)           # "R1"
print(robot2.name)           # "R2"
print(Robot.robot_count)     # 2 (dostępna na poziomie klasy)
print(robot1.robot_count)    # 2 (ale dostępna też z obiektu)

robot1.name = "R1_modified"  # Zmienia tylko robot1
robot1.robot_count = 10      # ✗ Tworzy NOWĄ zmienną instancji!
print(Robot.robot_count)     # Wciąż 2 – zmienna klasy się nie zmieniła
```

Best Practices – Zmienne klasy

- Używaj zmiennych klasy do przechowywania **wartości wspólnych** (np. kalibracja, limity).
- Zmienne instancji to **stan obiektu** (pozycja, prędkość, konfiguracja).

```
class Gripper:
    max_force = 100  # Wszystkie chwyty mają ten sam max

    def __init__(self, name):
        self.name = name
        self.current_force = 0  # Każdy chwyt ma inną siłę

gripper1 = Gripper("left_gripper")
gripper2 = Gripper("right_gripper")

# Zmiana wspólna
Gripper.max_force = 150
print(Gripper.max_force)      # 150
print(gripper1.max_force)     # 150 (dziedziczy z klasy)

# Zmiana lokalna
gripper1.current_force = 50
print(gripper1.current_force) # 50
print(gripper2.current_force) # 0 (niezmieniające)
```

Use-case: Robotyka

```
class Joint:
    """Reprezentuje joint (łącze) robota"""
    min_angle = -180    # Wspólny limit dla wszystkich jointów
    max_angle = 180

    def __init__(self, name, home_angle=0):
        self.name = name
        self.current_angle = home_angle
        self.is_calibrated = False

shoulder = Joint("shoulder")
elbow = Joint("elbow")
```

```
# Zmiana limitu dla całej klasy
Joint.min_angle = -170
Joint.max_angle = 170

shoulder.current_angle = 45  # Tylko dla shoulder
print(elbow.current_angle)   # 0 – elbow się nie zmienił
```

8.3 Dziedziczenie – Hierarchia klas

```
class Motor:
    """Klasa bazowa (parent)"""
    def __init__(self, name):
        self.name = name

    def spin(self, rpm):
        print(f"{self.name} spinning at {rpm} RPM")

class Servo(Motor):
    """Klasa potomna (child) – dziedziczy Motor"""
    def __init__(self, name, max_angle=180):
        super().__init__(name)  # Woła konstruktor rodzica
        self.max_angle = max_angle

    def move_to_angle(self, angle):
        if angle > self.max_angle:
            raise ValueError(f"Angle {angle} exceeds max {self.max_angle}")
        print(f"{self.name} moving to {angle}°")

# Użycie
servo = Servo("wrist_servo", max_angle=270)
servo.spin(100)           # Dziedziczy z Motor
servo.move_to_angle(90)   # Nowa metoda
```

Wielopoziomowe dziedziczenie

```
class Robot:
    def __init__(self, model):
        self.model = model

class HumanoidRobot(Robot):
    def __init__(self, model, num_fingers=5):
        super().__init__(model)
        self.num_fingers = num_fingers

class Atlas(HumanoidRobot):
    def __init__(self):
        super().__init__(model="Atlas", num_fingers=5)
```

```
    def walk(self):
        print("Atlas walking on two legs")

atlas = Atlas()
print(atlas.model)        # "Atlas"
print(atlas.num_fingers)  # 5
atlas.walk()              # Atlas walking on two legs
```

Best Practices – Dziedziczenie

- Używaj dziedziczenia gdy istnieje relacja "jest" (is-a). Servo **jest** Motor.
- Nie dziedzicz tylko żeby uniknąć pisania kodu – to prowadzi do `super()` hell.
- Głębia dziedziczenia: max 3–4 poziomy. Poza tym jest nieczytelne.

```
# × ANTYWZORZEC: Zbyt głębokie
class A: pass
class B(A): pass
class C(B): pass
class D(C): pass
class E(D): pass  # Kto wie, co ma E?

# ✓ DOBRZE: Jasna hierarchia
class Actuator: pass
class Motor(Actuator): pass
class Servo(Motor): pass  # Wyraźnie: Servo = Motor = Actuator
```

Use-case: Robotyka

```
class Actuator:
    """Bazowa klasa dla wszystkich napędów"""
    def __init__(self, name):
        self.name = name

    def power_on(self):
        print(f"{self.name} powered on")

class Motor(Actuator):
    """Motor elektryczny"""
    def __init__(self, name, max_rpm):
        super().__init__(name)
        self.max_rpm = max_rpm

class Servo(Motor):
    """Servo motor (wersja zaawansowana Motor)"""
    def __init__(self, name, max_rpm, max_angle):
        super().__init__(name, max_rpm)
        self.max_angle = max_angle
```

```
servo = Servo("wrist", max_rpm=60, max_angle=180)
servo.power_on()  # Dziedziczy z Actuator
```

8.4 super() – Bezpieczne wywoływanie metod rodzica

```
class Animal:
    def __init__(self, name):
        self.name = name

    def make_sound(self):
        return f"{self.name} makes a sound"

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # Woła Animal.__init__
        self.breed = breed

    def make_sound(self):
        parent_sound = super().make_sound()  # Woła Animal.make_sound()
        return f"{parent_sound}. Dog barks!"

dog = Dog("Buddy", "Golden Retriever")
print(dog.make_sound())
# Output: Buddy makes a sound. Dog barks!
```

Wielodziedziczenie (rzadko potrzebne, ale warto znać)

```
class Flyable:
    def fly(self):
        print("Flying")

class Swimmable:
    def swim(self):
        print("Swimming")

class Duck(Flyable, Swimmable):
    pass

duck = Duck()
duck.fly()   # Flying
duck.swim()  # Swimming

# Kolejność: szuka w Duck → Flyable → Swimmable → object
# To się zwie MRO (Method Resolution Order)
print(Duck.__mro__)
```

Best Practices – super()

- **Zawsze** używaj `super()` zamiast `ParentClass.__init__(self, ...)`.
- `super()` automatycznie dostosowuje się do wielodziedziczenia (MRO).
- Nieznane: `super(ClassName, self)` – stare, nie używaj w Pythonie 3.

```
# × STARY STYL (rigid, problematyczne przy wielodziedziczeniu)
class Child(Parent):
    def __init__(self):
        Parent.__init__(self)  # Wymaga znania nazwy rodzica

# ✓ NOWOCZESNY STYL
class Child(Parent):
    def __init__(self):
        super().__init__()  # Zmienia się automatycznie
```

8.5 Polimorfizm – Różne obiekty, wspólny interfejs

```
class Sensor:
    def read(self):
        raise NotImplementedError("Subclasses must implement read()")

class LidarSensor(Sensor):
    def read(self):
        return {"distance": 5.2, "angle": 45}

class CameraSensor(Sensor):
    def read(self):
        return {"image": "RGB", "resolution": (640, 480)}

class ImuSensor(Sensor):
    def read(self):
        return {"accel": [0.1, 0.2, 0.3], "gyro": [1, 2, 3]}

# Polimorfizm: każdy sensor ma `read()`, ale zwraca inne dane
sensors = [LidarSensor(), CameraSensor(), ImuSensor()]

for sensor in sensors:
    data = sensor.read()
    print(f"Sensor data: {data}")
```

Best Practices – Polimorfizm

- Definiuj wspólny interfejs w klasie bazowej (zwykle z metodami, które podnoszą `NotImplementedError`).
- Podklasy implementują konkretne zachowanie.
- Pozwala na łatwe dodawanie nowych sensorów bez zmiany kodu głównego.

```
# × BRAK POLIMORFIZMU (trudno dodawać nowe sensory)
if sensor_type == "lidar":
    data = lidar_read()
elif sensor_type == "camera":
    data = camera_read()
elif sensor_type == "imu":
    data = imu_read()

# ✓ POLIMORFIZM (łatwo dodawać)
for sensor in sensors:
    data = sensor.read()
```

8.6 Duck Typing – "If it walks like a duck..."

```
# Nie musisz dziedziczenia ani wspólnej klasy bazowej!
class Robot:
    def move(self):
        print("Robot moving")

class Car:
    def move(self):
        print("Car moving")

class Boat:
    def move(self):
        print("Boat moving")

# Wszystkie mają `move()` – to wystarczy
vehicles = [Robot(), Car(), Boat()]

for vehicle in vehicles:
    vehicle.move()  # Każdy zachowuje się inaczej

# Python nie sprawdza typów – patrzy na interfejs (co ma metod)
```

Best Practices – Duck Typing

- W Pythonie się unika sztywnych typów (isinstance checks) – to mniej Pythonic.
- Zamiast sprawdzać typ, sprawdź, czy obiekt ma potrzebne metody.

```
# × SZTYWNE
def process_sensor(sensor):
    if isinstance(sensor, LidarSensor):
        return sensor.read()
    elif isinstance(sensor, CameraSensor):
        return sensor.read()
    else:
```

```
        raise TypeError("Unknown sensor")

# ✓ PYTHONIC (duck typing)
def process_sensor(sensor):
    if hasattr(sensor, "read"):  # Czy ma metodę read()?
        return sensor.read()
    else:
        raise AttributeError("Sensor must have read() method")

# ✓✓ NAJLEPSZE (assume it works)
def process_sensor(sensor):
    return sensor.read()  # EAFP: Easier to Ask for Forgiveness than
Permission
```

8.7 Static methods (@staticmethod) – Funkcje bez `self`

```
class MathUtils:
    PI = 3.14159

    @staticmethod
    def distance(x1, y1, x2, y2):
        """Metoda statyczna – nie potrzebuje self ani instancji"""
        return ((x2 - x1)**2 + (y2 - y1)**2) ** 0.5

    @classmethod
    def from_dict(cls, data):
        """Metoda klasowa – otrzymuje klasę (cls) zamiast instancji"""
        return cls(data["name"])

# Używanie
dist = MathUtils.distance(0, 0, 3, 4)  # Nie trzeba tworzyć instancji!
print(dist)  # 5.0
```

Best Practices

- `@staticmethod` dla funkcji matematycznych, helperów, które nie potrzebują stanu.
- `@classmethod` dla alternative constructorów (factory pattern).

```
class Robot:
    def __init__(self, model, config_file):
        self.model = model
        self.config = load_config(config_file)

    @classmethod
    def from_yaml(cls, yaml_file):
        """Alternative constructor – wczytuj konfigurację z YAML"""
        data = load_yaml(yaml_file)
        return cls(model=data["model"], config_file=data["config"])
```

```
# Użycie
robot1 = Robot("Atlas", "config.yaml")
robot2 = Robot.from_yaml("robot.yaml")  # Czystszy interfejs
```

8.8 Magic methods (**str**, **repr**, **eq**)

```
class Motor:
    def __init__(self, name, rpm):
        self.name = name
        self.rpm = rpm

    def __str__(self):
        """Wywoływane przez str() i print() – czytelna reprezentacja"""
        return f"Motor({self.name}, {self.rpm} RPM)"

    def __repr__(self):
        """Wywoływane przez repr() – dla deweloperów"""
        return f"Motor('{self.name}', {self.rpm})"

    def __eq__(self, other):
        """Porównanie: == operatorem"""
        if not isinstance(other, Motor):
            return False
        return self.name == other.name and self.rpm == other.rpm

    def __lt__(self, other):
        """Porównanie: < operatorem"""
        return self.rpm < other.rpm

motor1 = Motor("Servo1", 100)
motor2 = Motor("Servo2", 150)

print(motor1)           # Motor(Servo1, 100 RPM)
print(repr(motor1))     # Motor('Servo1', 100)
print(motor1 == motor2) # False
print(motor1 < motor2)  # True
```

Best Practices – Magic Methods

- Zawsze definiuj `__str__` do debugowania.
- `__repr__` powinno być bardziej precyzyjne (dla kodu).
- `__eq__`, `__lt__` etc. pozwalają na naturalne porównania.

```
# ✗ BEZ MAGIC METHODS
motor = Motor("Servo1", 100)
print(motor)  # <__main__.Motor object at 0x...>
```

```
# ✓ Z __str__
print(motor)  # Motor(Servo1, 100 RPM)
```

8.9 @property – Gettery/settery bez metod

```
class Joint:
    def __init__(self, angle):
        self._angle = angle  # Private (konwencja: _ prefix)

    @property
    def angle(self):
        """Getter – wywoływane przez: j.angle"""
        print("Getting angle...")
        return self._angle

    @angle.setter
    def angle(self, value):
        """Setter – wywoływane przez: j.angle = value"""
        if value < -180 or value > 180:
            raise ValueError("Angle must be between -180 and 180")
        print(f"Setting angle to {value}")
        self._angle = value

    @angle.deleter
    def angle(self):
        """Wywoływane przez: del j.angle"""
        print("Deleting angle")
        self._angle = None

joint = Joint(90)
print(joint.angle)      # Getting angle... 90
joint.angle = 45        # Setting angle to 45
del joint.angle         # Deleting angle
# joint.angle = 270     # ValueError: Angle must be between -180 and 180
```

Best Practices – @property

- Używaj `@property` gdy chcesz dodać logikę do gettera/settera bez zmiany interfejsu.
- Prywatne zmienne prefiksem `_` (konwencja).
- Nie przesadzaj – proste atrybuty zostaw jako zwykłe zmienne.

```
# ✗ PRZESADA
class Robot:
    def __init__(self, x):
        self._x = x

    @property
    def x(self):
```

```
        return self._x

    @x.setter
    def x(self, value):
        self._x = value

# ✓ DOBRZE
class Robot:
    def __init__(self, x):
        self.x = x  # Zwykła zmienna, jeśli nie potrzebujesz logiki
```

8.10 Decorators (@decorator) – Modyfikacja funkcji

```
def time_it(func):
    """Dekorator mierzący czas wykonania"""
    import time

    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result

    return wrapper

@time_it
def move_robot(distance):
    """Symulacja ruchu"""
    import time
    time.sleep(1)
    return f"Moved {distance} meters"

result = move_robot(10)
# move_robot took 1.0021 seconds
# result = "Moved 10 meters"
```

Dekorator z parametrami

```
def repeat(times):
    """Dekorator powtarzający funkcję N razy"""
    def decorator(func):
        def wrapper(*args, **kwargs):
            results = []
            for _ in range(times):
                result = func(*args, **kwargs)
                results.append(result)
            return results
```

```
        return wrapper
    return decorator

@repeat(3)
def calibrate_sensor():
    return "Sensor calibrated"

print(calibrate_sensor())
# ['Sensor calibrated', 'Sensor calibrated', 'Sensor calibrated']
```

Best Practices – Decorators

- Dekoratory są potężne, ale mogą być trudne do zrozumienia.
- Używaj ich do walidacji, logowania, mierzenia czasu, cache'owania.
- Unikaj nadmiarowych zagnieżdżeń (max 2–3).

```
# ✓ PRAKTYCZNE ZASTOSOWANIA
@time_it
@log_execution
def send_command_to_robot(cmd):
    # Wykonaj komendę, zaloguj ją i zmierz czas
    pass

# × PRZESADA
@decorator1
@decorator2
@decorator3
@decorator4
def some_function():
    pass
```

Use-case: Robotyka

```
def safe_execution(func):
    """Dekorator do bezpiecznego wykonania komend robotycznych"""
    def wrapper(self, *args, **kwargs):
        try:
            if not self.is_connected:
                raise RuntimeError("Robot not connected")
            print(f"Executing: {func.__name__}")
            result = func(self, *args, **kwargs)
            return result
        except Exception as e:
            print(f"ERROR in {func.__name__}: {e}")
            self.emergency_stop()
            raise
    return wrapper

class Robot:
```

```
    def __init__(self):
        self.is_connected = True

    @safe_execution
    def move_forward(self, distance):
        print(f"Moving {distance}m")
        return f"Moved {distance}m"

    def emergency_stop(self):
        print("EMERGENCY STOP")

robot = Robot()
robot.move_forward(5)
```

---

9. EXCEPTION HANDLING – Obsługa błędów i stabilność

Koncept

Wyjątki (exceptions) to błędy, które mogą przerwać program. Obsługuj je zamiast pozwolić programowi "scrashować".

```
# BEZ OBSŁUGI – program się zamyka
data = [1, 2, 3]
print(data[10])  # IndexError: list index out of range
print("This won't print")

# Z OBSŁUGĄ – program kontynuuje
try:
    print(data[10])
except IndexError:
    print("Index out of range! Continuing...")
print("This will print")
```

Struktura: try-except-finally

```
try:
    # Kod, który może rzucić wyjątek
    value = int(input("Enter a number: "))
    result = 10 / value
except ZeroDivisionError:
    # Obsługa konkretnego błędu
    print("Cannot divide by zero")
except ValueError:
    # Obsługa innego błędu
    print("Input must be a number")
except Exception as e:
    # Łapanie WSZYSTKICH pozostałych błędów (ostatni resort)
    print(f"Unexpected error: {e}")
```

```
else:
    # Uruchamia się, jeśli NIE było wyjątku
    print(f"Result: {result}")
finally:
    # ZAWSZE się uruchamia (nawet przy return/raise)
    print("Cleanup done")
```

Hierarchia wyjątków (ważna!)

```
BaseException
├── SystemExit
├── KeyboardInterrupt
└── Exception  (ŁAP TĘ)
    ├── ValueError        # Złą wartość, np. int("abc")
    ├── TypeError         # Złą typ, np. "string" + 5
    ├── IndexError        # Indeks poza zakresem
    ├── KeyError          # Klucz nie w słowniku
    ├── FileNotFoundError # Plik nie istnieje
    ├── TimeoutError      # Timeout komunikacji
    ├── ConnectionError   # Błąd połączenia
    └── ... (setki więcej)
```

Best Practices – Exception Handling

- **Łap konkretne wyjątki**, nie generyczne `Exception`.
- **Nigdy** nie łap `BaseException` (w tym SystemExit, KeyboardInterrupt).
- Używaj `finally` do czyszczenia zasobów (zamykanie plików, rozłączanie).
- Umieszczaj wyjątek jako ostatnią opcję w łańcuchu `except`.

```
# × ANTYWZORZEC: Zbyt generyczne
try:
    sensor.read()
except:  # Łapie WSZYSTKO – nawet KeyboardInterrupt!
    pass

# ✓ DOBRZE: Konkretny wyjątek
try:
    sensor.read()
except TimeoutError:
    print("Sensor timeout – reconnecting...")
    sensor.reconnect()
except Exception as e:
    print(f"Unknown error: {e}")
```

Custom exceptions

```
class RobotError(Exception):
    """Bazowy wyjątek dla błędów robota"""
    pass

class MotorError(RobotError):
    """Motor nie odpowiada"""
    pass

class SensorError(RobotError):
    """Sensor zwrócił błędne dane"""
    pass

# Użycie
def spin_motor(motor_id, rpm):
    if rpm > 500:
        raise MotorError(f"Motor {motor_id} RPM limit exceeded: {rpm} >
500")

try:
    spin_motor(1, 600)
except MotorError as e:
    print(f"Motor problem: {e}")
except RobotError as e:
    print(f"General robot error: {e}")
```

Use-case: Robotyka (komunikacja z kontrolerem)

```
import socket

class CommunicationError(Exception):
    pass

def send_command_to_robot(ip, port, cmd):
    socket_obj = None
    try:
        socket_obj = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        socket_obj.settimeout(2.0)  # 2 sekund timeout
        socket_obj.connect((ip, port))

        socket_obj.sendall(cmd.encode())
        response = socket_obj.recv(1024).decode()
        return response

    except socket.timeout:
        raise CommunicationError("Connection timeout – robot not
responding")
    except ConnectionRefusedError:
        raise CommunicationError(f"Robot at {ip}:{port} refused
connection")
    except Exception as e:
```

```
        raise CommunicationError(f"Unknown communication error: {e}")

    finally:
        if socket_obj:
            socket_obj.close()  # ZAWSZE zamknij socket
            print("Socket closed")

# Użycie
try:
    response = send_command_to_robot("192.168.1.10", 5005, "MOVE 45")
    print(response)
except CommunicationError as e:
    print(f"Failed to communicate: {e}")
    # Fallback: offline mode, retry, etc.
```

---

10. FILE OPERATIONS – Odczyt i zapis danych

Koncept

Pliki to persistent storage. W robotyce: konfiguracje, logi, kalibracja.

```
# Otwarcie pliku
f = open("config.txt", "r")  # "r" = read, "w" = write, "a" = append
data = f.read()              # Czyta cały plik
f.close()                     # ZAWSZE zamknij!

# Problem: jeśli błąd, f.close() się nie wykona
# Rozwiązanie: with statement (context manager)
with open("config.txt", "r") as f:
    data = f.read()
# f.close() się wykonuje automatycznie, nawet przy błędzie
```

Tryby otwierania pliku

Mode	Opis	Użycie
`"r"`	Read (czytanie)	Domyślny; plik musi istnieć
`"w"`	Write (zapis)	Tworzy nowy lub ściera istniejący
`"a"`	Append (dołączanie)	Dodaje na koniec istniejącego pliku
`"x"`	Exclusive creation	Tworzy NOWY plik; błąd jeśli istnieje
`"b"`	Binary mode	`"rb"`, `"wb"` – dla obrazów, binariów

Odczyt

```python
with open("robot_config.txt", "r") as f:
    # Opcja 1: Czytaj wszystko na raz
    content = f.read()

    # Opcja 2: Czytaj linie (lista)
    lines = f.readlines()  # ['line1\n', 'line2\n']

    # Opcja 3: Czytaj linię po linii
    for line in f:
        print(line.strip())  # strip() usuwacze \n

    # Opcja 4: Czytaj jedną linię
    line = f.readline()  # Zwraca 'line1\n'
```

Zapis

```python
# Zapis (ściera plik, jeśli istnieje)
with open("log.txt", "w") as f:
    f.write("Robot started\n")
    f.write("Motor 1: OK\n")

# Dołączanie (dodaj na koniec)
with open("log.txt", "a") as f:
    f.write("Motor 2: OK\n")

# Zapis listy
sensor_readings = ["Temp: 25C", "Humidity: 60%", "Pressure: 1013hPa"]
with open("sensor_log.txt", "w") as f:
    f.writelines([line + "\n" for line in sensor_readings])
```

Best Practices – Files

- **Zawsze** używaj `with` statement – automatycznie zamyka plik.
- `.strip()` usuwanie białych znaków i `\n` z linii.
- Nie czytaj gigabajtowych plików naraz – czytaj chunkami.
- Sprawdzaj, czy plik istnieje przed czytaniem.

```python
import os

# Sprawdzenie istnienia pliku
if os.path.exists("config.yaml"):
    with open("config.yaml", "r") as f:
        config = f.read()
else:
    print("Config file not found – using defaults")
    config = DEFAULT_CONFIG

# Czytanie dużych plików (chunkami, nie na raz!)
```

```
with open("huge_sensor_log.txt", "r") as f:
    for line in f:  # Iteruje po liniach (lazy loading)
        process_line(line)

    # × NIE RÓB TEGO (duża pamięć)
    # all_lines = f.readlines()
```

Use-case: Robotyka (konfiguracja + logi)

```
import json
from datetime import datetime

class RobotLogger:
    def __init__(self, log_file="robot_log.txt"):
        self.log_file = log_file

    def log_event(self, event_type, message):
        """Dodaj event do logu"""
        timestamp = datetime.now().isoformat()
        with open(self.log_file, "a") as f:
            f.write(f"[{timestamp}] {event_type}: {message}\n")

    def read_recent_logs(self, lines=10):
        """Przeczytaj ostatnich N linii"""
        try:
            with open(self.log_file, "r") as f:
                all_lines = f.readlines()
                return all_lines[-lines:]
        except FileNotFoundError:
            return []

class RobotConfig:
    def __init__(self, config_file="robot_config.json"):
        self.config_file = config_file
        self.load()

    def load(self):
        """Wczytaj konfigurację z JSON"""
        try:
            with open(self.config_file, "r") as f:
                self.config = json.load(f)
        except FileNotFoundError:
            self.config = self._default_config()
            self.save()

    def save(self):
        """Zapisz konfigurację do JSON"""
        with open(self.config_file, "w") as f:
            json.dump(self.config, f, indent=2)

    def _default_config(self):
```

```
        return {
            "model": "Atlas",
            "max_speed": 10,
            "port": "/dev/ttyUSB0"
        }

# Użycie
logger = RobotLogger()
logger.log_event("STARTUP", "Robot initialized")
logger.log_event("MOTOR", "Motor 1 spinning at 100 RPM")

config = RobotConfig()
print(config.config)  # Odczyta z pliku lub użyje defaults

# Ostatnie 5 logów
recent = logger.read_recent_logs(5)
for log in recent:
    print(log)
```

---

11. DATES & TIMES – Czasomierze, timestampy, logi

Koncept

`datetime` to moduł do pracy z datami i godzinami. Niezbędny w logowaniu, scheduler'ach, pomiarach czasu.

```
from datetime import datetime, timedelta
import time

# Obecny czas
now = datetime.now()
print(now)  # 2026-01-26 15:35:20.123456

# Komponenty
print(now.year)       # 2026
print(now.month)      # 1
print(now.day)        # 26
print(now.hour)       # 15
print(now.minute)     # 35
print(now.second)     # 20

# Tworzenie daty
specific_date = datetime(2026, 5, 15, 14, 30, 0)
print(specific_date)  # 2026-05-15 14:30:00

# Różnica czasu (timedelta)
duration = timedelta(days=5, hours=2, minutes=30)
future = now + duration
print(future)  # Data o 5 dni, 2 godziny, 30 minut dalej

# Łączenie dat
```

```
start = datetime(2026, 1, 26, 10, 0, 0)
end = datetime(2026, 1, 26, 12, 30, 0)
elapsed = end - start
print(elapsed.total_seconds())  # 9000.0 (sekund)
```

Timestampy (UNIX time)

```
import time

# UNIX timestamp – liczba sekund od 1970-01-01
timestamp = time.time()
print(timestamp)  # 1737884120.123 (liczba zmiennoprzecinkowa)

# Konwersja: timestamp → datetime
from_timestamp = datetime.fromtimestamp(timestamp)
print(from_timestamp)  # 2026-01-26 15:35:20.123

# Konwersja: datetime → timestamp
to_timestamp = specific_date.timestamp()
print(to_timestamp)  # 1747502400.0
```

Formatowanie dat (strftime, strptime)

```
from datetime import datetime

now = datetime.now()

# Format do string'a (strftime)
formatted = now.strftime("%Y-%m-%d %H:%M:%S")
print(formatted)  # "2026-01-26 15:35:20"

# Parse string'a na datę (strptime)
date_str = "2026-05-15 14:30:00"
parsed = datetime.strptime(date_str, "%Y-%m-%d %H:%M:%S")
print(parsed)  # datetime(2026, 5, 15, 14, 30, 0)

# Popularne formaty:
# %Y = rok 4-cyfrowy, %y = rok 2-cyfrowy
# %m = miesiąc, %d = dzień
# %H = godzina (00-23), %M = minuta, %S = sekunda
```

Best Practices – Dates & Times

- Zawsze używaj `datetime` zamiast parsowania stringów ręcznie.
- Dla pomiarów czasu użyj `time.perf_counter()` (dokładny, niezależny od zegaru systemowego).
- W logach zawsze dołączaj timestamp dla możliwości debugowania.

```
# × ANTYWZORZEC
import time
start = time.time()
do_something()
end = time.time()
elapsed = end - start  # Wrażliwy na zmianę zegaru systemowego

# ✓ DOBRZE (dla pomiarów czasu)
import time
start = time.perf_counter()
do_something()
end = time.perf_counter()
elapsed = end - start  # Dokładny, niezależny od zegaru systemowego
```

Use-case: Robotyka (pomiary, harmonogram)

```
from datetime import datetime, timedelta
import time

class PerformanceMonitor:
    """Monitoruje czas wykonania operacji"""
    def __init__(self):
        self.measurements = {}

    def measure(self, operation_name):
        """Context manager do mierzenia czasu"""
        class Timer:
            def __init__(self, monitor, name):
                self.monitor = monitor
                self.name = name
                self.start = None

            def __enter__(self):
                self.start = time.perf_counter()
                return self

            def __exit__(self, *args):
                elapsed = time.perf_counter() - self.start
                if self.name not in self.monitor.measurements:
                    self.monitor.measurements[self.name] = []
                self.monitor.measurements[self.name].append(elapsed)

        return Timer(self, operation_name)

# Użycie
monitor = PerformanceMonitor()

with monitor.measure("move_forward"):
    time.sleep(0.5)  # Symulacja
```

```python
with monitor.measure("rotate_90"):
    time.sleep(0.3)

print(monitor.measurements)
# {'move_forward': [0.5001], 'rotate_90': [0.3001]}

class TaskScheduler:
    """Scheduler'uje zadania robotyczne"""
    def __init__(self):
        self.tasks = []

    def schedule(self, task_name, execution_time):
        """Zaplanuj zadanie na konkretny czas"""
        self.tasks.append({
            "name": task_name,
            "scheduled_at": execution_time
        })

    def run(self):
        """Wykonuj zadania gdy przychodzi czas"""
        while True:
            now = datetime.now()
            for task in self.tasks:
                if now >= task["scheduled_at"]:
                    print(f"Executing: {task['name']}")
                    self.tasks.remove(task)

            if not self.tasks:
                break
            time.sleep(1)

# Użycie
scheduler = TaskScheduler()
scheduler.schedule("calibrate_sensors", datetime.now() +
timedelta(seconds=5))
scheduler.schedule("move_to_home", datetime.now() + timedelta(seconds=10))
scheduler.run()
```

---

12. MULTITHREADING – Równoczesne czytanie sensorów i sterowanie

Koncept

Threading pozwala na równoczesne wykonywanie wielu zadań (I/O-bound: sieć, sensory, pliki). **Uwaga:** Python ma GIL (Global Interpreter Lock) – wielowątkowość nie działa dla CPU-bound operacji.

```python
import threading
import time

def sensor_reader(sensor_name):
    """Wątek czytający sensor"""
```

```
    for i in range(3):
        print(f"{sensor_name} reading: {i}")
        time.sleep(1)

# Tworzenie wątków
thread1 = threading.Thread(target=sensor_reader, args=("Lidar",))
thread2 = threading.Thread(target=sensor_reader, args=("Camera",))

# Uruchomienie
thread1.start()
thread2.start()

# Czekanie na zakończenie
thread1.join()  # Blokuje, aż thread1 się skończy
thread2.join()

print("All threads finished")
```

Thread vs Daemon

```
import threading
import time

def background_task():
    for i in range(5):
        print(f"Background: {i}")
        time.sleep(0.5)

def main_task():
    print("Main task starting")
    time.sleep(2)
    print("Main task done")

# Daemon = uruchamia się w tle, program nie czeka na jego koniec
daemon_thread = threading.Thread(target=background_task, daemon=True)
daemon_thread.start()

# Zwykły wątek = program czeka, aż się skończy
normal_thread = threading.Thread(target=main_task)
normal_thread.start()

normal_thread.join()  # Czekaj na main_task
print("Program ending")
# background_task będzie przerwane, bo jest daemon
```

Best Practices – Threading

- Używaj tylko do **I/O-bound** operacji (sieć, pliki, sensory), nie CPU-bound.
- Zawsze wywołuj `.join()` jeśli chcesz czekać na wątek.
- Unikaj **race conditions** – nie dziel zmiennych między wątkami bez synchronizacji.

```
# ✗ RACE CONDITION
shared_counter = 0

def increment():
    global shared_counter
    for _ in range(100000):
        shared_counter += 1

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)
t1.start()
t2.start()
t1.join()
t2.join()

print(shared_counter)  # Wynik: ~100000-200000 (nieprzewidywalny!)

# ✓ Z LOCK'iem (synchronizacją)
import threading

counter = 0
lock = threading.Lock()

def increment_safe():
    global counter
    for _ in range(100000):
        with lock:  # Tylko jeden wątek naraz może wejść
            counter += 1

t1 = threading.Thread(target=increment_safe)
t2 = threading.Thread(target=increment_safe)
t1.start()
t2.start()
t1.join()
t2.join()

print(counter)  # Wynik: zawsze 200000
```

Use-case: Robotyka (czytanie sensora + sterowanie)

```
import threading
import time
import queue

class RobotController:
    def __init__(self):
        self.sensor_data = queue.Queue()  # Thread-safe kolekcja
        self.running = True

    def sensor_reader_thread(self, sensor_name):
```

```
        """Wątek czytający sensor"""
        while self.running:
            # Symulacja czytania sensora
            value = sensor_name, time.time(), 10.5
            self.sensor_data.put(value)  # Thread-safe
            time.sleep(0.1)

    def control_loop_thread(self):
        """Główny wątek sterowania – czyta dane z sensora"""
        while self.running:
            try:
                sensor_name, timestamp, value =
self.sensor_data.get(timeout=1)
                print(f"Control: {sensor_name} = {value} at {timestamp}")
            except queue.Empty:
                pass

    def start(self):
        """Uruchom robota"""
        # Czytanie sensora w tle
        lidar_thread = threading.Thread(target=self.sensor_reader_thread,
args=("Lidar",), daemon=True)

        # Sterowanie główne
        control_thread = threading.Thread(target=self.control_loop_thread)

        lidar_thread.start()
        control_thread.start()

        time.sleep(5)  # Niech pracuje przez 5 sekund
        self.running = False
        control_thread.join()
        print("Robot stopped")

# Użycie
robot = RobotController()
robot.start()
```

---

13. BEST PRACTICES – Inżynierski kod

Struktura projektu

```
robot_controller/
├── main.py              # Entry point
├── config.py            # Konfiguracja
├── requirements.txt     # Zależności (pip install -r requirements.txt)
├── README.md            # Dokumentacja
├── hardware/
│   ├── __init__.py
│   ├── motor.py
```

```
│   ├── sensor.py
│   └── gripper.py
├── controller/
│   ├── __init__.py
│   └── robot_logic.py
├── utils/
│   ├── __init__.py
│   ├── logger.py
│   └── exceptions.py
└── tests/                # Testy jednostkowe
    ├── test_motor.py
    └── test_sensor.py
```

Dokumentacja

```python
def calculate_angle(x, y):
    """
    Oblicza kąt w stopniach na podstawie współrzędnych kartezjańskich.

    Args:
        x (float): Współrzędna X
        y (float): Współrzędna Y

    Returns:
        float: Kąt w stopniach (0-360)

    Raises:
        ValueError: Jeśli x == 0 i y == 0

    Example:
        >>> calculate_angle(1, 1)
        45.0
    """
    if x == 0 and y == 0:
        raise ValueError("Cannot calculate angle for (0, 0)")

    import math
    return math.degrees(math.atan2(y, x)) % 360
```

Naming conventions

```python
# Klasy: PascalCase
class MotorController: pass

# Funkcje, metody, zmienne: snake_case
def calculate_joint_angles(q1, q2): pass
sensor_reading = 5.2

# Stałe: UPPER_CASE
```

```
MAX_MOTOR_RPM = 500
PORT_TIMEOUT = 2.0

# Zmienne prywatne: _leading_underscore
class Robot:
    def __init__(self):
        self._internal_state = None  # Prywatna
        self.public_attribute = 42     # Publiczna
```

Error handling (defensive programming)

```
def move_robot(robot, target_x, target_y):
    """Move robot to target position with validation"""

    # Validacja
    if not isinstance(robot, Robot):
        raise TypeError("robot must be Robot instance")

    if not isinstance(target_x, (int, float)) or not isinstance(target_y,
(int, float)):
        raise TypeError("Coordinates must be numbers")

    # Bounds checking
    if target_x < -100 or target_x > 100:
        raise ValueError(f"X coordinate {target_x} out of bounds [-100,
100]")

    try:
        robot.move_to(target_x, target_y)
    except ConnectionError as e:
        logger.error(f"Failed to move robot: {e}")
        robot.emergency_stop()
        raise  # Re-raise, aby caller wiedział o błędzie
```

Logging (zamiast print)

```
import logging

# Setup
logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)

handler = logging.FileHandler("robot.log")
formatter = logging.Formatter("%(asctime)s - %(levelname)s - %(message)s")
handler.setFormatter(formatter)
logger.addHandler(handler)

# Użycie
logger.debug("Debug info – szczegóły")
```

```
logger.info("Information – normalne operacje")
logger.warning("Warning – potencjalny problem")
logger.error("Error – coś nie działa")
logger.critical("Critical – system w niebezpieczeństwie")

# Zamiast
# print("Motor started")  ×
# logger.info("Motor started")  ✓
```

---

14. QUICK REFERENCE – Ścieżka wykonania

```
# 1. STRUKTURY DANYCH
iterable_obj = [1, 2, 3]
for item in iterable_obj: pass

# 2. MEMBERSHIP
"key" in dict_obj
"value" in collection

# 3. COMPREHENSIONS
squared = [x**2 for x in numbers]
filtered = [x for x in numbers if x > 5]

# 4. MATCH-CASE
match state:
    case "idle": pass
    case "moving": pass
    case _: pass

# 5. MODULES & IMPORTS
from module import function

# 6. SCOPE (LEGB)
# Local → Enclosing → Global → Built-in

# 7. if __name__ == "__main__"
if __name__ == "__main__":
    main()

# 8. OOP
class Robot:
    def __init__(self, name):
        self.name = name

    def move(self):
        pass

# 9. INHERITANCE
class Servo(Motor):
    def __init__(self, name, max_angle):
```

```
        super().__init__(name)
        self.max_angle = max_angle

# 10. EXCEPTIONS
try:
    risky_operation()
except SpecificError:
    handle_error()
finally:
    cleanup()

# 11. FILES
with open("file.txt", "r") as f:
    content = f.read()

# 12. DATETIME
from datetime import datetime
now = datetime.now()

# 13. THREADING
import threading
thread = threading.Thread(target=func, daemon=False)
thread.start()
thread.join()
```

Podsumowanie

Ta ściąga obejmuje wszystkie koncepty z kursu Bro Code kluczowe dla inżynierii robotyki. Każdy temat ma:

- **Koncept** – co to jest i dlaczego ważne
- **Kod** – praktyczne przykłady
- **Antywzorce** – co unikać
- **Use-case** – zastosowanie w robotyce

Drukuj, czytaj, praktykuj. Gdy zaczniesz Projekt 1, będziesz mieć referencję pod ręką.

Gotowy do Projektu 1? Wrzuć OK, a podpowiadamy!